

1. Introduction

1.1 Topic and research question

Football is one of the most commercialized sports in the world, attracting billions of dollars of investment every year, and the market value of players has become one of the key indicators of its commercial potential and influence. In addition to market value, a player's performance on the field (goals, assists, etc.) is also an important factor in assessing their contribution. Although the relationship between market value and player performance has received much attention, the complexities involved have not been fully studied. The factors that affect market value are not limited to on-court performance; a player's visibility in the public and media also significantly increases their market value. Understanding these factors is crucial for investors, clubs and fans alike.

The core of this study is to explore the relationship between players' market value and their performance on and off the field, focusing on the impact of performance and popularity on market value. For investors, accurately predicting a player's market value will help them assess their financial potential, optimize their return on investment, and avoid risk. By identifying the link between media exposure and market value, investors are able to more precisely assess a player's commercial potential. For clubs, mastering this relationship not only improves player performance, but also allows them to make more informed decisions when managing financial resources. Understanding the interaction between market value and player popularity helps clubs maximize the financial return of their players through sponsorship, merchandising, etc. Fans can gain a fuller understanding of what constitutes a player's value and make informed decisions to support a team or player.

Overall, this research provides key insights for investors, clubs and fans to help optimize decisions. Investors will be able to invest more efficiently, clubs will be able to manage player assets better, and fans will enjoy the game on a more informed basis. This comprehensive perspective not only reveals the complexity of the football economy, but also provides a valuable basis for future strategy formulation.

1.2 Dataset

This dataset covers the multidimensional characteristics of football players, helping us to fully understand player performance and market value. Here's a breakdown of the data and the challenges:

This dataset covers the multidimensional characteristics of football players, helping us to fully understand player performance and market value. Here's a breakdown of the data and the challenges:

1. Feature classification:

- (1) General characteristics: such as player ID, name, country, club, etc., provides the basic information of the player.
- (2) Performance characteristics: such as rating, goals, assists, etc., to help us evaluate the performance of players on the field.
- (3) Popular characteristics: such as hot shots, reflecting the player's popularity outside the court.
- (4) Financial characteristics: such as market value, weekly salary, release terms, etc., are directly related to the financial status and market performance of players, and are the core objectives of our research.
- (5) Physical characteristics: such as height, weight and age, provide information about the physical status of the player.

2. Challenges:

- (1) Missing data: Certain features, such as "loan end date", only apply to a small number of players. If these missing values are not handled properly, the accuracy of the model may decrease. Therefore, how to fill these vacancies reasonably without introducing bias is crucial.
- (2) Layout problems: Although we have cleaned up the data, there may still be inconsistencies in classification features such as "location". Models need to uniformly encode these features in order to be able to efficiently process and understand different types of data.
- (3) Class imbalance: The distribution of market value is highly uneven, and the market value of a few players is much higher than that of other players. To combat this, we use equal-frequency classification, which divides the market value of players into 10 categories and ensures that there are roughly the same number of players in each category. This balanced classification method effectively avoids the bias of the model to the low-value players, so that the model can identify the high-value players better.
- (4) Bias: Certain traits, such as "popularity," reflect public perception more than actual player performance. Over-reliance on such subjective characteristics can lead to bias in model predictions. In addition, a player's "sweet spot" is highly correlated with his "actual position," which can make the model over-rely on certain features and ignore other important variables.

(5) Too many features: the large number of features in the data set may lead to overfitting or underfitting problems in the model. Therefore, selecting and processing features reasonably is the key to model performance.

1.3 Setup

In our setup we discussed as a team our target attribute 'Market value' as Categorized range, as well as discussed every single attribute meaning features and how we can statistically subgroup it without ignoring its effect on our prediction and machine learning model. We also discussed our validation model and what is our success criteria measure.

1.3.1 Modelling agreements

The first phase focuses on identifying the target data fields for predictions and assessing their impact in real-world scenarios. It is clear that many of the numerical attributes in the dataset show strong correlations. In the course of the discussion, we focus on the potential value of predictive models in practical applications.

At this stage, we also discuss whether a categorical model or a continuous model is more suitable for the prediction of market value. We recognize that player evaluation involves subjective factors and external influences that can lead to significant outliers if relying solely on a continuous model. In addition, many disaggregated data points, such as club names, geographic locations, and management decisions, are crucial in determining a player's market value. These factors can bias forecasts in a continuous model. Therefore, we conclude that using a classification model, the nuances of player valuations can be better captured by dividing predictions into predefined ranges. This decision is also supported by the transfer pricing structure, which often follows clear decision-making criteria (for example, highly rated players from established clubs in a certain age range may be sold at a higher value).

Once the appropriate model type and prediction fields have been identified, we begin to select evaluation metrics. We believe that the following metrics best align with our research objectives and provide a robust assessment of the predictive model:

The F1 score combines accuracy and recall to measure the overall performance of the model when dealing with unbalanced data sets. In player market value forecasting, since the sample size of high-value players is usually small, F1 scores help us balance the model's performance across these categories. With F1 scores, we are able to ensure that the model captures the majority of high-value players (high recall rate) while reducing the number of cases where it mispredicts high-value players (high accuracy rate). When predicting market value, overestimating or underestimating the value of a player can cause serious financial errors for a club, so F1 scores are particularly important to ensure efficient and accurate decision making. A high F1 score means that the model performs consistently in identifying high-value players, providing a reliable basis for clubs to predict.

The recall rate reflects the model's ability to identify all relevant instances, especially in the identification of high-value players. For clubs, missing out on any of their high-value players could result in missing out on lucrative transfer opportunities. The high recall rate means that the model is able to successfully capture the majority of players who should be classified as high-value, avoiding missing potentially important targets. In the player market value forecast, the increase in recall rate helps to ensure that the club has a comprehensive understanding of the high-value players in the market, helping them not to miss any important acquisition opportunities in the fierce market competition. Therefore, the recall rate is decisive for the model's performance in identifying high-value players.

ROC-AUC scores measure a model's ability to distinguish between different categories by assessing its performance at different thresholds. The higher the AUC, the better the model is at distinguishing between positive and negative classes. In market value forecasting, the ROC-AUC score is particularly important because clubs may adjust the criteria for judging the market value of players under different transfer Windows, budget constraints or strategic priorities. It can effectively evaluate the performance of the model under different market conditions and ensure that the model still has a strong discriminative ability under different thresholds. The high ROC-AUC score representation model has a strong ability to distinguish between high-value and low-value players, and can provide a basis for clubs to make accurate financial decisions in the transfer market.

1.3.2 Data division

We decided to split the data into a training set of 90% and a test set of 10%, which may be adjusted based on the preferences of team members and the algorithms used. Because some members use the k-fold cross-validation algorithm with built-in hyperparameter tuning, the validation set is adjusted differently according to the specific algorithm.

2. Model (Random Forest)

2.1 Predictive model

2.1.1 Model description

Name: Random Forest

I decided to use Random Forest as my model.

Random Forest is an assembled learning method based on decision tree, by combining multiple trees together to enhance the performance and robustness. It has certain underlying assumptions:

- **Independence of data**

Random Forest assumes the all datapoints are independent. This is generally not strictly holds for real world data. But in our dataset, one player's data are hardly severely impacted by another one. Although this does not strictly hold since they are all playing the same game, we may assume the correlation are quite small and won't be an issue. Also, in Random Forest, the random sampling and independent building trees for each step would also alleviate this problem.

- **Same distribution**

Random Forest assumes the all datapoints share a same distribution. We can say that this will hold in our dataset. Since football is arguably the most commercialized sport and we are basically studying an economic relation in this game, we can say this relation will holds for all players, because if not, the invisible hand of the market will fix this issue almost immediately, via, for example, buying and selling the player. So, this assumption holds.

- **Class Balance**

Random Forest assumes the all class are balanced. We can say that this will hold in our dataset. As described in the group section, this issue is addressed by using equal frequency binning. So, our dataset holds for this.

- **Independence of feature**

Strictly speaking, Random Forest requires that the features to be independent. This is generally not strictly holds for real world data. In our dataset, for most of the features, this holds, for example, one's scoring capability is independent to his defense. But for some, this does not hold. Luckily, in most of the case, Random Forest only requires relative independence of features, we can achieve this by hand picking out the most troublesome features and the Random Forest can lower the risk of overfitting through random selection of features.

Random Forest, from experience and from information from a TA of STAT5003, is one of the best performing models in the context of traditional learning methods. Since it is an assemble method, Random Forest can output great performance using weak basic classifiers. Its high accuracy even under complicated dataset making it exceptionally suitable for this task. Also, due to it assemble nature, Random Forest has remarkably robustness for noise and outliers. Additionally, it can automatically assess the importance of features, identifying the factors that most significantly impact model performance, thus aiding in model optimization. Lastly, Random Forest performs well in handling missing values, effectively managing instances of partial data absence.

All merits come at costs. In Random Forest, the most significant drawback is its computational cost. Because Random Forest constructs multiple decision trees, the computational time and memory usage can be considerable, especially with larger datasets. Moreover, compared to a single decision tree, Random Forest's interpretability is near to none, making it impossible to directly understand how each feature affect the final results. Lastly, when dealing with class imbalances, Random Forest favor predictions for more common classes, resulting in inadequate performance for minority classes. Luckily, our dataset is not beyond the scope for personal computer and the class imbalance issue was long solved by me at the very beginning. As for interpretability, well, most of the best performing model suffers from it and I decided to leave it as it is.

All these justify its suitability for the research question and dataset characteristics.

2.1.2 Model algorithm

The idea behind the Random Forest is to assemble a lot of weak decision trees together, by averaging the result, to improve its performance and robustness.

Basic principle

- **Decision Tree:** Random Forest is based on decision tree. By using different attributes to classify data, Decision Tree minimize the impurity of the data, commonly measured by entropy.
- **Assemble:** With DECISION TREE in hands Random Forest assemble those together through multiple iterations, while introducing randomness in this process. For each iteration
- Random Forest will using bootstrap method to resample the dataset and train a Decision Tree based on it.
- Random Forest will perform a random feature selection, and train the Decision Tree based on this subset of feature.

Hyperparameters

Normally, Random Forest contains the following hyperparameters:

- `n_estimators`: describe the amount of Decision Tree built in the final model, usually the bigger the number, the better the performance and higher computation cost
- `max_depth`: describe the max depth of each Decision Tree, used to prevent overfitting
- `min_samples_split`: describe the minimal amount of sample require to split a node, used to prevent overfitting
- `min_samples_leaf`: describe the minimal amount of sample require for a leaf node node, used to prevent overfitting
- `max_features`: describe the maximum number of features to consider when looking for the best split. This parameter affects the randomness of the selected features.

Pseudocode

1. Initialize Random Forest

Input:

- training data
- hyperparameters

Output: Random Forest model

2. Build Random Forest

For $i = 1$ to `n_estimators`:

- Create subset `i` of training data using bootstrap sampling
- Create decision tree `tree_i` using subset `i`:
 - Initialize the root node of the tree
 - While `max_depth` is not reached:
 - For each node:
 - Randomly select `max_features` features
 - Calculate impurity for each feature
 - Select the feature with the highest information gain for splitting
 - Split the data into child nodes
 - Store `tree_i` in the random forest

3. Predict using the Random Forest:

Input: test data

Output: final predictions

For each instance in `test_data`:

- Initialize an empty list predictions
- For each tree in the random forest:
 - Predict the class of the instance using the tree
 - Append the prediction result to predictions
- For classification:
 - `final_prediction` = Majority voting of predictions

Return `final_predictions`

2.1.3 Model development

The process begins in data preprocessing.

First, several irrelevant features were removed to avoid unnecessary complexity. Features such as `PlayerName`, `PlayerID`, `Photo`, and `URLs` were eliminated as they have no impact on a player's performance or market value. Removing these non-informative features helped streamline the model by focusing only on relevant data.

Second, to prevent data leakage, key financial features like `Weekly Wage`, `Release Clause`, and `Market Value` were excluded from the dataset. These features are directly related to the player's financial status and could lead to overfitting if included in predictions of market value. Excluding them ensures that the model does not use information that would boost its performance but reduce its generalization ability.

Additionally, features unrelated to a player's performance or marketability, such as `Date Joined` and `Loan End Date`, were removed. These features do not contribute meaningfully to predictions about a player's on-field performance or public perception, and their inclusion might introduce temporal biases. Therefore, their exclusion ensures the model focuses on more meaningful features.

Also, feature engineering is applied.

A new feature reflecting the contract status of players was introduced. This feature shows whether a player was free, on loan, or on contract, adding valuable context about the player's availability.

Finally, change the encoding and deal with NaNs.

Three additional features capturing the region of the player's country, the country of their club, and the region of their club's country were added. These geographical features may influence a player's marketability or performance based on regional scale. By introducing these factors, the model gains insight into regional effects that could impact the target outcomes.

To manage categorical variables, the Playing Positions feature was transformed using one-hot encoding via MultiLabelBinarizer and all others are using label encoding via LabelEncoder.

Missing values were handled using imputation. NaN values only appears in features like Shooting and Popularity Hits and were replaced with zeros. Since the missing values shows no shooting or popularity hits rather than missing data due to an error, imputing them as 0 was a logical choice. This approach ensures that missing values do not skew the model's performance.

Hyperparameters Selection

Several hyperparameters were tuned to optimize model performance.

The `n_estimators`, being the most important one, which controls the number of decision trees, was set to 100. This value achieves a balance between computational efficiency and model performance. Increasing the number of trees improves the model's performance, but after a certain point, the returns diminish, so 100 trees were selected to ensure stable predictions.

The `max_depth` was limited to 10 to avoid overfitting. Trees with unlimited depth tend to learn the noise in the data, resulting in poor generalization on new data. By restricting the depth, the model remains more focused on the main patterns in the data.

The `min_samples_leaf`, set to 3, ensures that each leaf node in the decision trees contains at least three samples. This prevents overly specific splits that would cater too much to the training data and ensures that the splits are statistically meaningful.

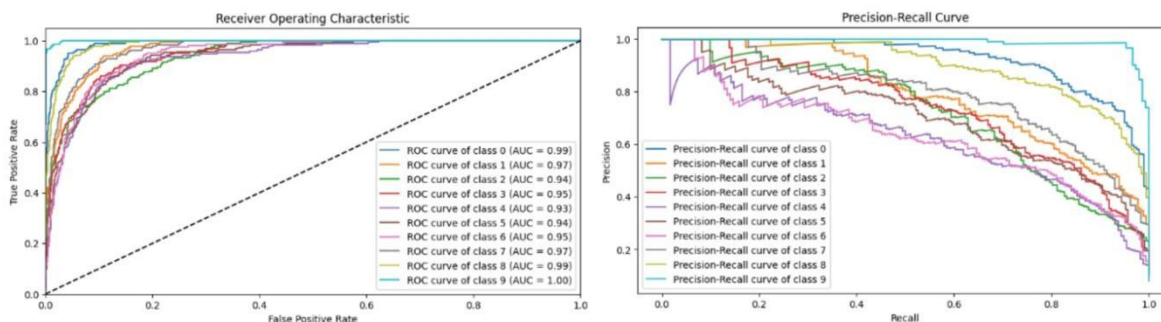
Similarly, the `min_samples_split` was set to 5, meaning that a node must have at least five samples before it can be split further. This ensures that the model does not create too many unnecessary splits.

Last, as a ritual, `random_state` of 42 was set to ensure that the model's results are reproducible, and for better luck.

In terms of implementation, the `RandomForestClassifier()` function from the `sklearn.ensemble` library was used.

2.2 Model evaluation and optimization

2.2.1 Model evaluation



To evaluate the performance of the model, we used several agreed metrics including F1 Score, Accuracy, Precision, Recall, and ROC-AUC Score. The results showed that the model achieved a F1 Score of 0.7060, an Accuracy of 0.7139, Precision of 0.7348, and Recall of 0.7139. These results show that the model is relatively balanced in terms of its ability to correctly identify relevant instances (precision) and retrieve them (recall). However, the F1 score being slightly lower suggests that there is room for improvement in balancing precision and recall.

Furthermore, the ROC AUC score of 0.9615 is particularly noteworthy as it demonstrates that the model is excellent at distinguishing between different classes.

This is reinforced by the ROC curve (displayed in the first image), where the curves for the majority of the classes are close to the top-left corner, indicating strong performance. The ROC AUC scores for the individual classes (ranging from 0.93 to 1.00) further validate this, showing that the model is effective across various class labels.

The Precision-Recall curve (shown in the second image) provides additional insight into the performance of the model. While precision is relatively high, the decline in the precision-recall curve for some classes suggests that the model

struggles with a few specific categories. This could be attributed to noise or the imbalance in the dataset, impacting both recall and precision in these cases.

Overall, the evaluation metrics suggest that the model performs well in general but could benefit from further optimization. Additionally, fine-tuning hyperparameters and incorporating more advanced feature engineering may help enhance performance across all metrics.

In terms of interpretability, while the random forest model provides reliable results, understanding the decision-making process for each prediction is nearly impossible due to the nature of the complexity of the ensemble learning technique.

2.2.2 Model optimization

To optimize the performance of the model, I employed both grid search and gradient boosting.

First, I used grid search with a 3-fold cross-validation to tune the hyperparameters. After running the grid search, the best parameters were identified. Results are as follow: max_depth: None; min_samples_leaf: 1; min_samples_split: 2; n_estimators: 50.

These settings resulted improved the model's overall performance, with the following metrics:

F1 Score of 0.7543, Accuracy of 0.7555, Precision of 0.7573, Recall of 0.7555, and a ROC-AUC Score of 0.9713.

These improvements show that the grid search effectively tuned the hyperparameters, enhancing its ability to classify more accurately and reducing both false positives and false negatives. The high ROC-AUC score shows that the model maintains a strong ability to distinguish between classes.

In the next step, I further optimized the model by implementing gradient boosting to build a new classifier based on the improved random forest with the best hyperparameters gathered from grid search.

Gradient boosting is another ensemble learning technique that by focusing on correcting the errors made by previous models by sequentially building new models to improve the performance.

With this, final result was achieved: Final Accuracy of 0.8751, Final F1 Score of 0.8750, Final Recall of 0.8751, Final Precision of 0.8753, and a Final ROC-AUC Score of 0.9927.

The nearly perfect ROC-AUC score shows that the model now performs exceptionally well when distinguishing between different classes.

Overall, the gradient boosting approach succeeded in refining the classifier, and the optimized model now delivers significantly better results compared to the initial one, particularly in terms of both general accuracy and f1.

3. Model (Decision Tree)

3.1 Predictive Model

3.1.1 Model description

Name: Decision Tree

For this project, I chose decision trees as the core model for predicting player market prices. Decision tree is a tree-structured supervised learning. Decision tree may divide a dataset into different categories or predicting continuous values by recursively partitioning the feature space (Aurelien Geron 2022). It generates rules with high interpretability by selecting features that best distinguish the target variables, allowing the model to predict the market value in a simple way.

The decision tree model has several core assumptions:

1. The decision tree assumes that there is regularity between features and target variables. The dataset can be divided into regular categories through feature-based conditional judgment. Therefore, it is very suitable for the prediction of the market value of a player, because the characteristics of a player's height, weight, country, club may affect their market value.

2. By selecting the optimal feature, the decision tree divides the data set into multiple subsets and assumes that after multiple splits, the data can be divided into relatively pure subsets, i.e. all players in the leaf nodes fall into the same market value range. This method is particularly effective for player classification and market value prediction, because the key attributes of players (such as position, ability value, etc.) are strongly differentiated in different market value ranges.

3. In addition, the splitting process of the decision tree is based on local decisions, and only the features and data of the current node are considered each time, which means that each feature can make the optimal decision independently of other features when splitting, thus improving the flexibility and accuracy of the model.

Applicability evaluation

Decision trees have **significant advantages** in predicting the market value of players: First, decision trees are interpretable, each segmentation is based on specific characteristics, and can clearly show the key factors affecting the value of players, which is particularly useful for club managers and scouts. Secondly, the decision tree does not need to scale features, and can directly process numerical and category features, simplifying data preprocessing. In addition, decision trees can capture the non-linear relationship between market value and characteristics, which is suitable for dealing with complex market fluctuations. However, **the decision tree also has limitations**: it is prone to overfitting, especially when the depth of the tree is large, and may not generalize well to new data; Sensitive to small changes in data, which may lead to instability of the tree structure; In addition, avoiding overfitting often requires pruning trees and adjusting hyperparameters, increasing the complexity and computational cost of the model.

Overall, decision trees provide an intuitive way to process complex multidimensional feature Spaces and capture the contribution of individual features to market value. For example, "country", "current club", "contract information", etc., these characteristics often have a clear hierarchical relationship. By segmenting feature space step by step, decision tree can effectively reveal the relationship between these factors.

3.1.2 Model algorithm

In this project, I choose decision tree as the basic model. The following parts introduce the algorithm principle of decision tree and how to divide subsets through decision tree.

3.1.2.1 Decision Tree

A decision tree is an algorithm that classifies a data set recursively by splitting it into subsets, starting from the root node by selecting the best features according to the splitting criteria (such as information entropy or Gini index), and gradually building a rule set to classify the data. Each internal node represents a feature, each branch represents a value of that feature, and each leaf node represents the final classification value. Therefore, the execution step of the decision tree is to first select the best features by splitting criteria such as information gain or Gini index, so as to maximize the purity of the data set. Then, as the data continues to be split, the decision tree gradually generates a tree-like structure until all the data points belong to the same category as far as possible on the leaf nodes, or some preset stopping condition is reached, such as the maximum tree depth or the minimum number of sample splits.

Next, we introduce two different split criteria for decision trees and how to construct a basic decision tree model.

3.1.2.2 Gini index

According to Aurelien Geron (2022), the smaller the value of the Gini index, the more "pure" the samples in the node are, that is, most of the samples belong to the same category. A larger Gini index means that the samples in the node are more mixed and contain multiple categories. The formula is as follows:

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

G_i : Indicates the Gini index of node i , which measures the impurity of the node.

$p_{i,k}$: represents the proportion of samples belonging to class k in node i .

n : indicates the number of categories on node i .

The Gini index is calculated by subtracting the sum of the squares of probabilities for all categories from 1. The closer the value is to 0, the higher the purity of the node is. The closer the value is to 0.5 (in the binary classification problem), the higher the impurity of the node, indicating that the sample distribution in the node is more uniform

3.1.2.3 Information entropy

According to Aurelien Geron (2022), in the decision tree algorithm, Gini impurity is used as the measure of splitting by default, but entropy can be selected as the measure of impurity by setting the hyperparameter criterion to "entropy". Entropy represents the degree of molecular disorder, and here it means that when the classified data belong to a class of labels, the smaller the entropy value, the better the splitting effect. The formula is:

$$H_i = - \sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^n p_{i,k} \log_2(p_{i,k})$$

n : indicates the number of categories, that is, the number of different categories. For a classification task, n represents the number of possible categories

$p_{i,k}$: represents the proportion of instances belonging to class k in node i , i.e. the probability of class k in the current node i

The probability of each type is added together to obtain the information entropy, and the smaller the value of information entropy, the better the effect of splitting according to this type

3.1.2.4 Model construction

Using Scikit-learn (version: 1.4.2), DecisionTreeClassifier was introduced to build a basic tree model:

```
tree = DecisionTreeClassifier(criterion='entropy',
                             max_depth=15,
                             min_samples_split=5,
                             min_samples_leaf=10,
                             random_state=42)
tree.fit(X_train, y_train)
```

1. `max_depth`: indicates the maximum depth of the tree. To avoid overfitting.
2. `criterion`: Gini index and information gain (entropy)
3. `min_samples_split`: indicates the minimum number of samples required for re-splitting internal nodes.
4. `min_samples_leaf`: indicates the minimum number of samples contained in a leaf node.

These hyperparameters search for the optimal parameters in the grid search to build the optimal decision tree in the search range. The pseudo-code is as follows:

```
# Function to build a decision tree from a dataset
def BuildTree(dataset):
    """
    Recursively builds a decision tree from the input dataset.
    Parameters:
    - dataset: A collection of samples, where each sample has features and a class label.
    Returns:
    - A decision tree node (either a leaf node with a class label or a root node with branches).
    """
    # Base case: If all samples in the dataset belong to the same class
    if all_samples_in_same_class(dataset):
        # Return a leaf node with that class label
        return create_leaf_node(get_class_label(dataset))
    # Otherwise, continue building the tree
    else:
        # Step 1: Select the best feature to split the dataset
        best_feature = select_best_feature(dataset)
        # Create a root node for the current split
        root = create_root_node(best_feature)
        # Step 2: For each unique value v of the selected feature
        for value in unique_values_of_feature(best_feature, dataset):
            # Step 3: Create a branch for this value
            branch = create_branch(root, value)
            # Step 4: Split the dataset based on the feature value v
            subset = split_dataset(dataset, best_feature, value)
            # Step 5: Recursively build the tree for the subset (branch)
            child_node = BuildTree(subset)
            # Step 6: Attach the child node (subtree or leaf) to the current branch
            attach_child_node(branch, child_node)
        # Step 7: Return the root node, which now has branches and child nodes
        return root
```

3.1.3 Model development

Although in this project the data has been basically preprocessed, I go a step further with Label Encoding to convert the string characteristics (e.g., Country, CurrentClub, etc.) to numeric form. For decision trees, numerical data is easier to

split because they can be directly divided according to their eigenvalues to form different branch structures. To ensure consistency in the data, I also converted features such as height and weight, standardizing them into uniform centimeters (cm) and kilograms (kg). Such processing makes the eigenvalues consistent across different quantities and helps the decision tree model converge more quickly. Next, I used the `train_test_split` function to split the dataset into a 90% training set and a 10% test set. Doing so ensures that the model can be validated on previously unseen data to assess its ability to generalize.

After the above data feature engineering, we trained a single decision tree based on the initial Settings. `GridSearchCV` is used for hyperparameter grid search. By using cross-validation to test the performance of the model under different combinations of hyperparameters, the optimal combination of hyperparameters is found, and the best single decision tree is obtained. In order to further improve the generalization ability of the model and reduce the overfitting problem that may occur in a single decision tree, Bagging is introduced at last. Bagging enhances the generalization ability of the model by integrating multiple optimal decision trees, generating multiple models and voting their predicted results. Next, the basic principles of each algorithm are described in detail, including the optimization process of grid search, and Bagging's integration method.

3.2. Model evaluation and optimization

3.2.1 Model evaluation

In this research problem, the prediction of a player's market value relies on the interaction of several characteristics, such as player ability, contract information, and past performance. These characteristics are weighted differently in different categories, such as high market value players versus low market value players.

By evaluating the performance of the basic decision tree model, we find that its F1 score is 0.7807 and its accuracy is 0.7803. Although the model as a whole can capture most of the relationships between features, it performs poorly in some categories (such as class 3 and class 4). This may reflect a class imbalance in the dataset, where there are fewer samples of high market value players and more samples of middle and low market value players, resulting in insufficient ability of the model to distinguish between a few categories. The ROC AUC of 0.81 and 0.82 in these categories, respectively, is much lower than the model's performance in other categories, such as class 0 and class 9, where the AUC is close to 0.95 and 0.98, further supporting the negative effect of class imbalance on model performance.

In addition, noise in the data set is also an important factor affecting the performance of the model. Changes in a player's market value may be affected by short-term factors or external events, such as media coverage, transfer rumours or injuries, noise that may not be adequately expressed or reflected in the data. In the precision-recall curve of the basic decision tree model, we notice that the Precision and Recall rates of some categories are low (such as class 4), indicating that the model may be disturbed by noise when processing these categories, resulting in more missed and false positives. By adapting feature engineering, processing noisy data, and using stronger integration methods such as Bagging, it is possible to effectively reduce the interference of these noises to the model, enhance the robustness of the model, and improve the classification accuracy in complex scenarios.

3.2.2 Model optimization

In this study, the hyperparameters of the decision tree model are optimized by using Grid Search, and the generalization ability of the model is further improved by using Bagging. The optimization process involved searching for key hyperparameters such as `max_depth`, `min_samples_split`, `min_samples_leaf`, and `criterion`, which significantly improved the performance of the model.

3.2.2.1 Grid Search

Grid search is a hyperparameter optimization technique, which is used to search multiple hyperparameter combinations by exhaustive method to find the optimal parameter Settings. The basic principle is to first define a parameter grid, list all possible combinations of hyperparameters, then train the model under each combination and evaluate the performance of each combination using cross-validation techniques. Finally, the optimal parameter combination is selected. For example, if `max_depth` has three values and `min_samples_split` has three values, there are a total of hyperparameter combinations, and the grid search tries each of the nine combinations in turn, eventually choosing the best hyperparameter setting. The advantage of grid search is that it can ensure to find the global optimal parameter combination, but because of its exhaustion method, the calculation cost is high, especially when the hyperparameter space is large, the time complexity and the consumption of computing resources may be very high. Therefore, grid search is suitable for smaller parameter Spaces, or combined with parallel computing to speed up the search process.

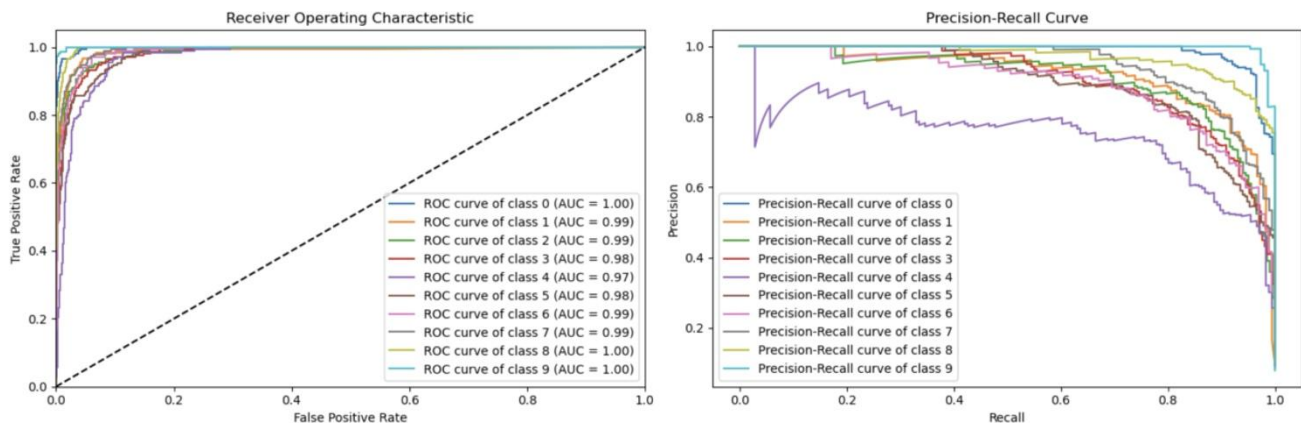
The decision tree model optimized by grid search shows obvious performance improvement. The F1 score has improved from 0.7807 to 0.8015 for the initial model, which means that the model is more stable in terms of balancing accuracy and recall. The accuracy rate was increased from 0.7803 to 0.8019, indicating that the model can be more accurate in the overall classification. The accuracy rate reached 0.8022, showing the model's enhanced ability to reduce

false positives, while the recall rate was 0.8019, indicating that the model was able to capture more positive samples. The ROC AUC also reached 0.9502, especially the AUC near 0.99 on class 0, 1, and 9, indicating that the model's ability to differentiate in these categories is almost perfect.

3.2.2.2 Bagging

Bagging (Bootstrap Aggregating) is a classical ensemble learning method that trains multiple models by generating multiple different training subsets and integrating their predictions. The main step is to randomly sample the original data set with retractions, generate a different subsample set each time from the original data set, and then train an independent model (usually a high-variance model, such as a decision tree) on each subsample set. Because each subset may contain duplicate samples, these models produce different predictions when trained. Bagging's final prediction is made by integrating the output of all models: a vote is taken in the classification task, and the predictions are averaged in the regression task. Its core advantage is that by integrating the results of multiple models, Bagging can effectively reduce the variance of a single model and significantly improve the generalization ability of the model, resulting in more stable performance on the test set. The main limitation of Bagging is that it does not reduce the bias of the model. If the underlying model itself has a large bias, such as an underfit problem, Bagging cannot effectively improve this phenomenon. Since Bagging improves performance mainly by reducing variance, the effect is limited for problems with high model deviation.

After Bagging integrated optimization, the performance of the model is further improved. The F1 score increased from 0.8015 to 0.8522, indicating that the integrated approach effectively improved the predictive balance of the model. The accuracy has been improved from 0.8019 to 0.8519, indicating that the accuracy of the integrated model in the classification task has been further optimized. The accuracy rate was increased to 0.8529, further reducing false positives in the model, and the recall rate was 0.8519, indicating that the model captured more positive samples. After integration, the ROC AUC reached 0.9887, indicating that the integrated model showed very high differentiation ability in all categories, especially in class 0, class 1 and class 9, the AUC value reached 1.00, showing very high classification accuracy. The Bagging model produces the following images:



3.2.3 conclusion

As can be seen from the figure, the AUC value of class 0, class 8 and class 9 is 1.00, indicating that the classification effect of these classes is almost perfect; class 1, class 2, class 6, and class 7 have AUC values close to 0.99, and the classification effect is also very close to perfect. Although the AUC values for class 3, class 4, and class 5 are slightly lower, they are still between 0.97 and 0.98, indicating that the model is also highly discriminating in these categories.

The precision-recall curve is mainly used to deal with class imbalance, and the Precision and Recall rates perform well in most categories, especially class 0, class 1, class 8, and class 9. However, the low recall rate of class 4 indicates that the model has an underreporting problem in this category, and the accuracy rate of class 5, class 3, and class 7 also decreases, indicating that the model's predictions are unstable in these categories.

In summary, the combined application of grid search and Bagging makes the model more stable and efficient when processing complex and noisy data. Although the performance of the model has improved significantly in most categories, there are a few categories (such as class 4) that perform slightly less well, so future work can continue to explore other integration methods (such as Boosting) or further optimize data preprocessing and feature engineering to further improve the model's balance and performance across all categories. With these improvements, the model will be more applicable and able to more accurately predict the market value of players and meet real business needs.

4. Model (logistic regression)

4.1 Predictive Model

4.1.1 Model description

The football player price dataset includes 65 columns with attributes such as Country, Age, Rating, Club, Contract Info, and skills like Dribbling and Goalkeeping. It also reflects performance through aggregate stats like Total Stats and Popularity Hits. However, there are biases—players from big clubs or younger players are often overvalued, while defenders and goalkeepers tend to be undervalued. I applied normalization by position and attempted normalization by club using revenue and country data to reduce these biases. Through analysis, I identified key factors influencing market value: popularity, overall stats (especially for midfielders), position (attackers valued higher than defenders), and age (mid-aged players valued more). For instance, players from the British Premier League rank higher, while outliers like Messi and Ronaldo maintain high prices due to their fame despite being in less competitive leagues. Specific stats were also particularly important for goalkeepers.

4.1.2 Model algorithm

4.1.2.1 Cleaning Phase (data reduction)

I iterated over the dataset's attributes, clustering data by positions and using metrics like ROC AUC and confusion matrices. I chose to divide player positions into three categories: Goalkeepers (GK), Defenders (CB, LB, RB, LWB, RWB), and Attackers & Midfielders (ST, LW, RW, CDM, LM, RM, CAM). This positional clustering helped refine my model's accuracy, showing how different factors impact pricing based on the player's role. I used KMeans clustering grouping data points into K clusters based on feature similarities, using centroids to represent each cluster's center. Euclidean distance is typically used to measure closeness between points and centroids in the algorithm itself. The optimal number of clusters (K) is determined using methods like the Elbow Method and Silhouette Score. I applied KMeans using Python's scikit-learn. For evaluation, I used ROC AUC to handle imbalanced classes, ensuring a comprehensive performance overview. The F1 Score was crucial due to the unequal consequences of false positives and negatives, ensuring balanced precision and recall. I also used a confusion matrix and ROC curve to assess accuracy. After a thorough review, I kept only relevant attributes but believe the model can still be improved.

4.1.2.2 Actual Prediction Algorithm

I used Logistic Regression predicts the probability of an event occurring based. I won't put the equation here. In One-vs-Rest, we need to separate binary classifiers and the highest predicted probability is chosen. Logistic regression is simple to implement, provides probabilistic outputs, and works well in many classification tasks. However, it assumes a linear relationship between features and log-odds, struggles with imbalanced data, and is primarily designed for binary classification, which may require adaptations for multi-class problems. That's why I divided the data into training 85%, evaluation 5% and testing 10%.

4.1.3 Model development

Cleaning phase (data reduction)

I already discussed my data reduction technique yet I will try to highlight the important python code functions I used, I hope my naming is well identifying what is meant, I also put some comments on the code, yet may be some are outdated.

CodeSight

`validateAttributesWithPosition` main function used description below

First it scales the data for better performance during clustering then uses KMeans algorithm to the selected attributes as input to see if they group up with specific type, changes the label of `PlayerPosition` to true or false based on the input column values. If value sit in one of these then it goes to true otherwise it goes to false. It then groups the data by cluster and checks for consistency via roc visual. I then use `validate_grouping()` to check if specific attributes tend to fall into certain clusters because I have two class resulted from my clustering code, yet it might be wrong and I want to see that in my output. The output also includes the Confusion Matrix report which I print along with `validate_grouping` function.

4.1.3.1 Logistic regression

I used to scikit-learn once again **Logistic Regression** with a few binary classifier

CodeSight

Missing Value Handling: I addressed missing values in the popularity column by imputing zeros, representing no popularity.

Binning Numerical Data: Discretization, or binning, was applied to convert continuous numerical data into discrete ranges. This process reduces noise and simplifies calculations.

One-Hot Encoding: Categorical columns were preprocessed using one-hot encoding, which converts categorical data into binary vectors, making it suitable for machine learning algorithms.

Defining Target and Data Splitting: The dataset was split using the `train_test_split` function, allocating 90% of the data for training and validation, and 10% for testing. To prevent bias, the 90% was further split into 85% for training and 5% for validation.

Model Training: Logistic Regression was trained on the 85% training set.

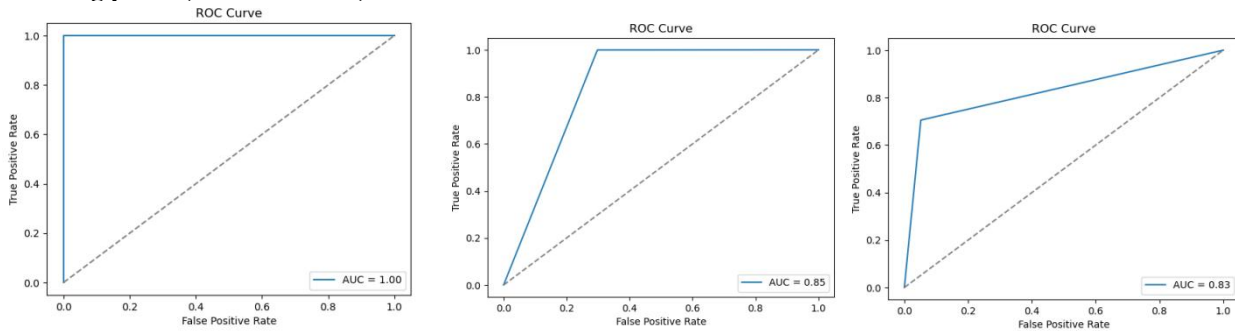
Model Evaluation: The model was evaluated using the ROC curve, first on the 5% validation set and then on the 10% test set. Comparing the ROC values ensured no significant performance drop.

Coefficient Representation: Finally, the model's coefficient values were represented in the user interface.

4.2 Model evaluation and optimization

4.2.1 Model evaluation

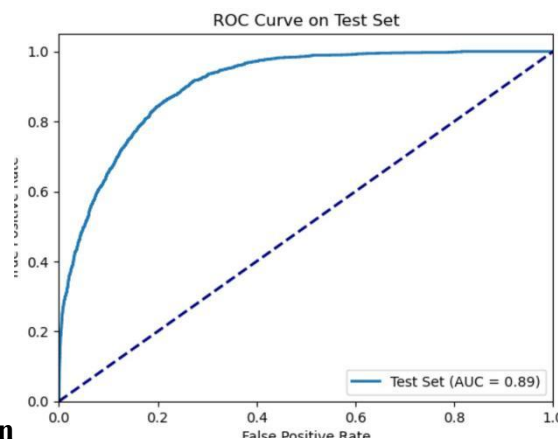
Cleaning phase (data reduction)



The ROC results show that the model performs well for goalkeepers and defenders but struggles with attackers, likely due to low attribute measures and disparities in roles (e.g., midfielders with weaker attacking skills). While the model distinguishes between classes effectively (AUC = 0.83), it underperforms in predicting the True class, with a recall of 0.75 and an F1-score of 0.79, possibly due to the unequal distribution of attributes between defenders and attackers. This led to data reduction to Key attributes selected for prediction include **Age**, **Rating (Scale 100)**, **Potential (Scale 100)**, **Current Club**, and various technical and physical skills, Contract-related information, like **LoanEndDate** and popularity metrics.

Logistic Regression Evaluation

The roc for evaluation set was 0.9 while the roc for the test set was 0.89 which means I have a good level of coherence in my results I also have a high roc which means the model might work good for evaluating players market value. This seemed like good enough.



4.2.2 Model optimization

Cleaning phase (data reduction)

I have iterated over various attributes to ensure that only important attributes are left into calculation. The code as it is now is a result of multiple iteration effort

Logistic Regression

As described above I went again over many steps and have binned many data as well as increased the number of iterations to improve the prediction model it is set now to 200.

4.2.3 Conclusion

We were able to have a high confidence Machine Learning algorithm using **One-vs-Rest Logistic Regression** which is create we were also able to identify noises and handle data efficiently and we got really good prediction result. We can improve by adding club income and/or League annual return on investment might have been used instead of just the club's name

5.1 Discussion

5.1.1 Random Forest

Random forests are a powerful ensemble learning method that enhances model accuracy by aggregating the predictions of multiple decision trees. Their main advantage lies in the ability to effectively handle large datasets and high-dimensional features, while being less prone to overfitting compared to single decision trees, which demonstrates high reliability in classification tasks. Additionally, random forests provide feature importance analysis, enhancing the model's interpretability.

However, random forests also have some significant limitations. Firstly, the complexity of the model may hinder the understanding of prediction results, making it difficult to trace the decision path for specific instances, thereby reducing interpretability. Secondly, random forests have high demands on computational resources, and training time may increase significantly with the number of trees and the size of the dataset.

On the other hand, gradient boosting focuses on correcting the errors made by previous models to enhance performance. This iterative approach is very effective in improving accuracy as it gives more weight to previously misclassified instances. However, due to its sequential nature, it also requires more computational resources and training time.

In summary, while random forests have advantages in robustness and handling various data types, they fall short in interpretability and resource requirements. Gradient boosting, on the other hand, provides higher accuracy and performance but may also require more resources.

5.1.2 Decision Tree

The biggest advantage of the decision tree is that it is highly interpretable, and each split node is based on specific feature conditions, which can clearly show which features are most important for the prediction of market value. This provides intuitive explanations in practical applications, especially for club managers and scouts, thus helping to understand the decision logic behind the model. In addition, decision tree does not require complex data preprocessing, it can directly deal with numerical and categorical data, without feature scaling, which greatly simplifies the process of data preparation. Furthermore, decision trees are also good at capturing complex non-linear relationships in the data, for the complex relationship between market value and player characteristics, the decision tree can be split multiple times to identify the interaction between different characteristics.

However, decision trees also have limitations. First, overfitting is one of the main problems with decision trees, especially when the depth of the tree is large, the model may be too sensitive to noise in the training data, resulting in unsatisfactory performance on the test set. To deal with this problem, it is often necessary to prune the tree or use a regularization strategy. In addition, decision trees are very sensitive to small changes in the data, and even small changes in the training data can lead to large changes in the structure of the tree, which can affect the stability of the model. Finally, the limitation of the decision tree model itself is that when it is close to optimal, further performance improvement becomes difficult. Thus, while decision trees perform well in player market value prediction, they may need to be combined with other ensemble learning methods such as Bagging or random forests to further improve performance when dealing with high data complexity or uneven data distribution.

Overall, the team considers that the application of decision trees in this project is reasonable and effective, especially its high interpretability and nonlinear processing capabilities make it perform well in predicting the market value of players, but at the same time, its limitations need to be considered and properly optimized in practical applications.

5.1.3 logistic regression

A major advantage of logistic regression models is their simplicity and interpretability. Because it is a linear model, logistic regression provides an easy to understand and interpret way to assess the impact of each feature on the outcome. For example, when predicting the market value of a player, logistic regression can help us clearly see the positive or negative impact of a particular characteristic (such as the player's age or club) on the market value. In addition, the fast training and low computational overhead of logistic regression also make it suitable for processing larger data sets. For some relatively simple scenarios, such as considering only a few key factors that affect market value (such as a player's age, position, and performance), logistic regression may be effective enough.

The biggest limitation of logistic regression is that it can only capture linear relationships in the data, whereas in complex problems (such as the prediction of a player's market value) there are often non-linear relationships between features. For example, the market value of a player is not only linearly affected by a single factor, but is influenced by the interaction of multiple complex factors, such as the interaction between a player's on-court performance and his popularity. In addition, logistic regression models are difficult to deal

with the problem of class imbalance. For example, in a market value forecast, high market value players may have far less data than low market value players, and logistic regression will most likely favor the more common low market value category, ignoring the characteristics of high value players.

5.2 Conclusion

After comparing the performance of decision tree, random forest and logistic regression models, random forest is chosen as the best model. Although the performance of the decision tree model is quite excellent, with F1 score of 0.8522, accuracy rate of 0.8519, accuracy rate of 0.8529, recall rate of 0.8519 and ROC AUC of 0.9887, these indicators indicate that it has a good performance in handling complex classification tasks, compared with random forest, There are still some gaps.

In contrast, logistic regression, while having similar accuracy rates and recall rates, has limited processing power for nonlinear relationships, which makes it inferior to decision trees and random forests when dealing with complex data. Random forest effectively copes with the nonlinear and noise problems in the data through ensemble learning technology, and further enhances the prediction ability of the model through gradient lifting optimization, especially in the key index of ROC-AUC, which performs particularly well, showing the strong ability of the model to distinguish different categories. Therefore, based on these data and evaluation results, the Random forest model is considered to be the most suitable model for predicting the market value of players.

The stochastic forest significantly improves the overall performance of the model by integrating multiple decision tree learning methods. After gradient lifting optimization, the final performance of the random forest model is particularly outstanding, with F1 score of 0.8750, accuracy rate of 0.8751, accuracy rate of 0.8753, recall rate of 0.8751 and ROC-AUC score of 0.9927. In contrast, random forest outperforms decision trees in almost all key indicators, especially in the ROC-AUC score, indicating that it can not only handle nonlinear relations, but also improve the generalization ability of the model by reducing overfitting. Therefore, random forest is considered to be the best model to predict the market value of players.

Reference

1. Aurélien Géron 2022, Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow, 3rd edn, O'Reilly Media, Inc. (Aurélien Géron 2022)
2. Charbuty, B & Abdulazeez, A 2021, 'Classification Based on Decision Tree Algorithm for Machine Learning', Journal of Applied Science and Technology Trends, vol. 2, no. 01, pp. 20–28. (Charbuty & Abdulazeez 2021)